IPAS 2026
The 5th International Practical Applications Seminar on Electrical Engineering and Computer Science

PerfRunbench: When Is Runtime Tuning Worthwhile?

Author: Ding-Xu Wu Advisor: Prof. Shan-Hsiang Shen
National Taiwan University of Science and Technology | Department of Electrical Engineering

01 Why test server settings?

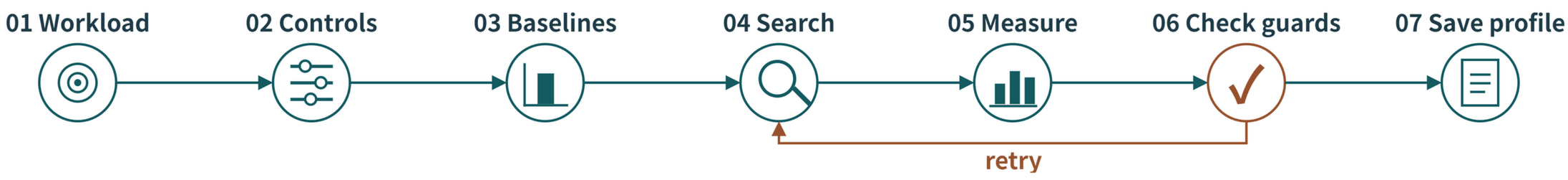
Imagine asking a local LLM a question while other CPU-intensive jobs run on the same computer. The LLM must share processing time with those jobs, so a configuration that looks fast in a quiet test may take longer to produce its first word or finish its answer. PerfRunbench compares runtime settings on matched workloads, measures both output rate and response delay, and saves the chosen configuration along with the time spent finding it. The practical question is whether that advantage still holds in fresh tests when the machine remains busy and more requests arrive at once.

Block = one randomized pass through all compared settings.

02 What changed across separate studies?

CPU sharing	***04*** compares shared and reserved CPU allocations and measures LLM tokens/s and completed jobs/s.
Inference screen	***05*** screens seven server settings and compares their output-rate effects.
Response delay	***06*** checks response delay at 1, 4 and 8 simultaneous users.
Agent + chat	***07*** compares file-reading tasks and chat while CPU jobs run.
Training payback	***08*** uses a separate T4g training example. Search paid back at job 30.

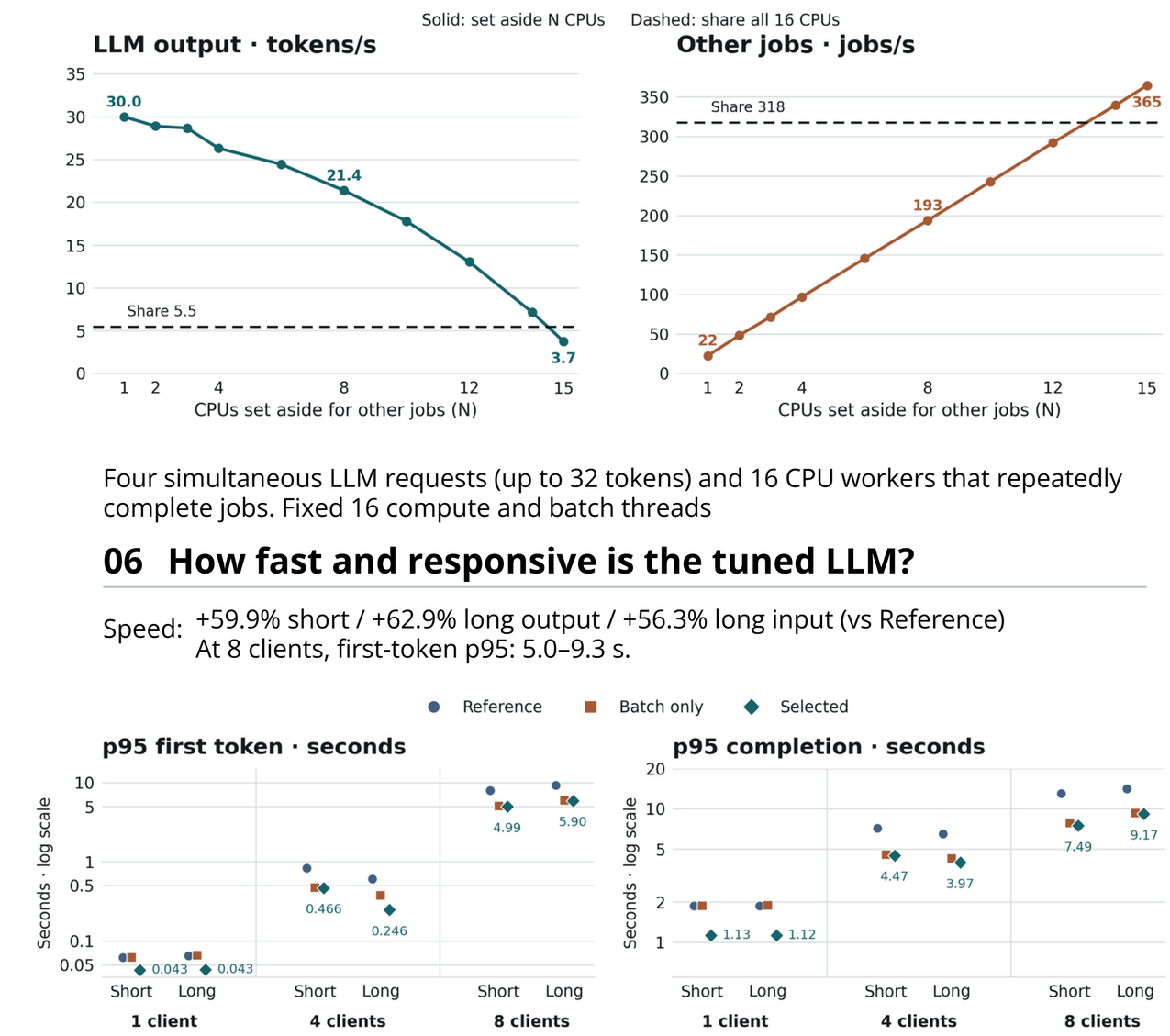
03 How to test and choose settings



04 How should I split 16 CPUs?

Share : the LLM and 16 CPU workers may use any of the 16 CPUs
Set aside : N CPUs go to workers; the LLM uses the other 16−N

At N = 8, LLM output rose from 5.5 to 21.4 tokens/s. Other jobs fell from 318 to 193 jobs/s.

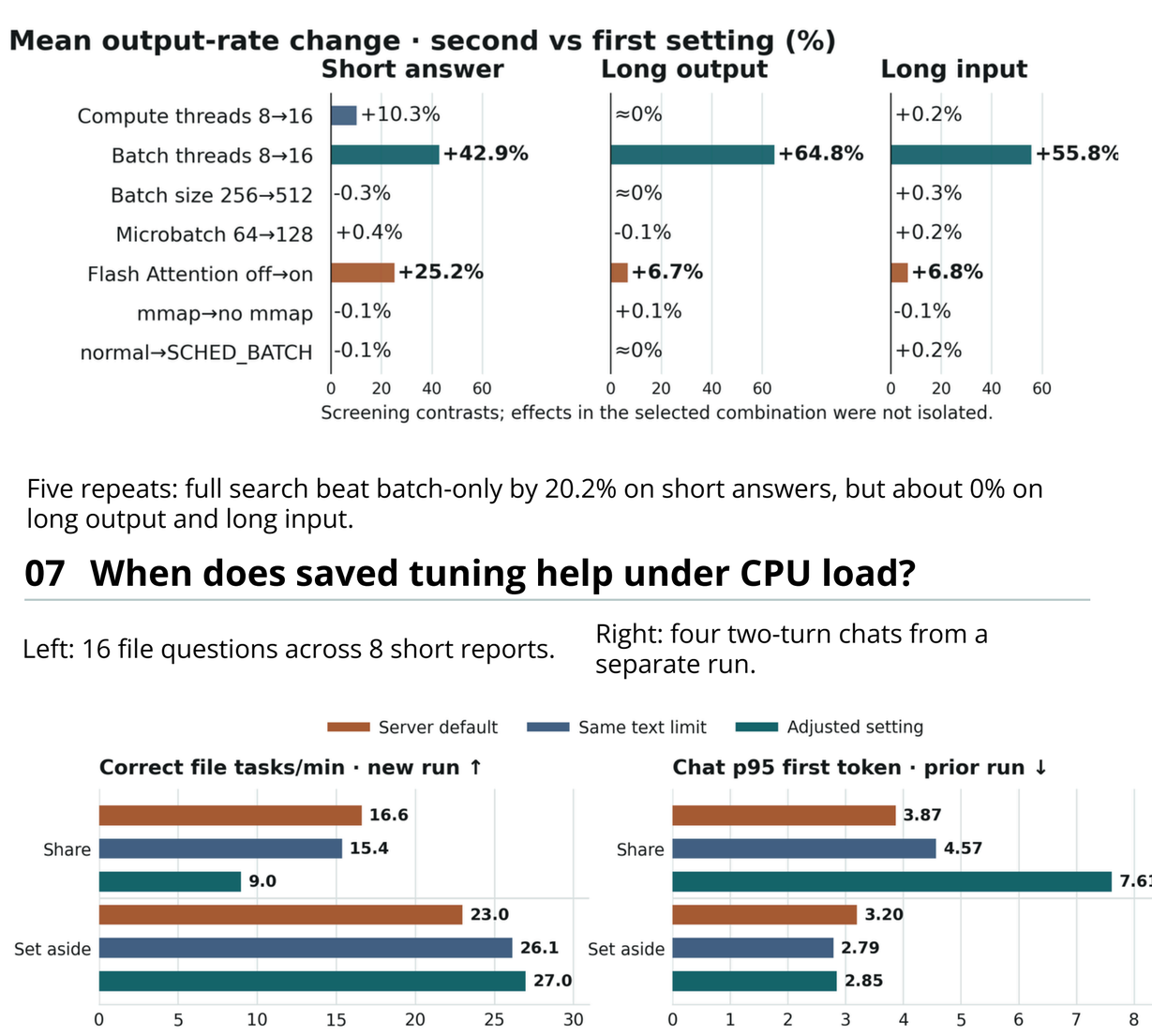


Four simultaneous LLM requests (up to 32 tokens) and 16 CPU workers that repeatedly complete jobs. Fixed 16 compute and batch threads

05 Which server controls matter most?

I screened 32 combinations in two orders (1,408 requests). Each row shows the output-rate change from the first to second setting.

Batch threads had the largest average gain. Flash Attention averaged +25.2% for short answers. Its effect in the chosen combination was not isolated, and that combination kept it off.



Five repeats: full search beat batch-only by 20.2% on short answers, but about 0% on long output and long input.

06 How fast and responsive is the tuned LLM?

Speed: +59.9% short / +62.9% long output / +56.3% long input (vs Reference)
At 8 clients, first-token p95: 5.0–9.3 s.

Dots = median p95 over five fresh repeats of the same six request workloads.

07 When does saved tuning help under CPU load?

Left: 16 file questions across 8 short reports.

Right: four two-turn chats from a separate run.

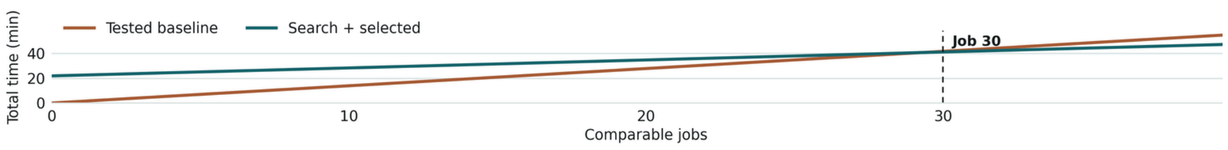
Left: 16 indexed-file tasks in one randomized block. Each got 15/16 correct.

Right: separate five-block chat run. Set aside = 8+8 CPUs.

08 Does the time spent searching pay back?

Training: 82.99 → 38.39 s per comparable job. Search 1,306 s.

Break-even: 1,306 ÷ 44.60 = 29.3 → job 30.



09 Saved settings need a fresh check for each task

perfrunbench run --auto --preset quick -- python train.py

I wrap an existing command and apply a confirmed profile on later runs. Separate inference and C++ build workflows search their own settings. The saved record links the command, versions, measurements and search cost.

The wrapper tunes exposed controls. train.py must expose internal batch size. Recheck correctness, speed and delay at real load. These are controlled examples.

TAIWAN TECH
NATIONAL TAIWAN UNIVERSITY OF SCIENCE AND TECHNOLOGY
Pusan National University
College of Engineering